College of Computing and Data Science

Academic Year 2025/2026

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC4003 Intelligent Agents

Submitted by: Sally Ngui Yu Ying

Instructor: Dr. Zhang Jie

Contents

1 Part 1: Given Maze Environment **1**

1.1 Problem Description 1

1.1.1 Description of Implemented Solution 2

1.1.2 Plot of Optimal Policy 3

1.1.3 Plot of Utilities of All States 4

1.1.4 Plot of Utility Estimates Against Number of Iterations 6

1.2 Policy Iteration 7

1.2.1 Description of Implemented Solution 7

1.2.2 Plot of Optimal Policy 9

1.2.3 Plot of Utilities of All States 10

1.2.4 Plot of Utility Estimates Against Number of Iterations 11

1.3 Comparison of Results 13

2 Part 2: Complicated Maze Environments **14**

2.1 Problem Description 14

2.2 Methodology 14

2.3 Experiments 15

2.3.1 Custom Mazes 15

2.3.2 Fixed-Size Wall Density Experiments 19

2.3.3 Fixed-Density Size Scaling Experiments 23

2.4 Conclusion 25

1 Part 1: Given Maze Environment

1.1 Problem Description

This section studies the given maze environment (Figure 1). The agent can move **Up, Down, Left, or Right**. The intended outcome occurs with probability 0.8, and with probability 0.1 the agent moves at either right angle to the intended direction. If the move would make the agent walk into a wall, the agent stays in the same place as before. There are no terminal states, hence the agent's state sequence is infinite. In this section, we find the optimal policy and the utilities of all the (non-wall) states using both **Value Iteration** and **Policy Iteration**.

The rewards are:

- White cell: -0.05
- Green cell: $+1$
- Brown cell: -1

The discount factor used is $\gamma = 0.99$.

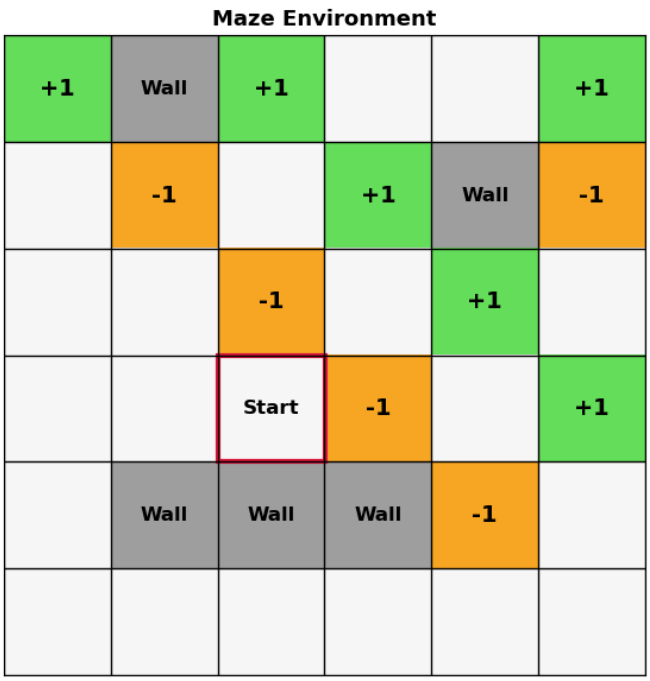


Figure 1: Maze Environment

1.1.1 Description of Implemented Solution

Value Iteration begins with initialising the utilities of all states as zero. The algorithm updates the utility of each non-wall state simultaneously by iteratively applying the Bellman optimality equation until convergence.

In this implementation, the reward is attached to the **next state entered**. The update step in the algorithm is:

$$U_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[R(s') + \gamma U_k(s') \right] \tag{1}$$

where:

- $U_k(s)$ is the utility of state s at iteration k
- $P(s' \mid s, a)$ is the transition probability
- $R(s')$ is the reward of the next state
- γ is the discount factor, which is 0.99 in this question

Once the maximum absolute utility change across all states falls below the predefined threshold, the algorithm is said to have converged. This method is guaranteed to converge to the optimal utility function as $\gamma < 1$. The threshold is set at 1×10^{-8}, and the maximum iterations is set at $10,000$.

1.1.2 Plot of Optimal Policy

Figure 2 shows the optimal policy obtained using Value Iteration. The optimal policy shows a clear preference for actions that move the agent towards green states with the highest reward, while avoiding unnecessary exposure to brown states with the lowest reward. The agent tends to move upward or leftward at most states, highlighting that the upper-left region is higher in value in the long run.

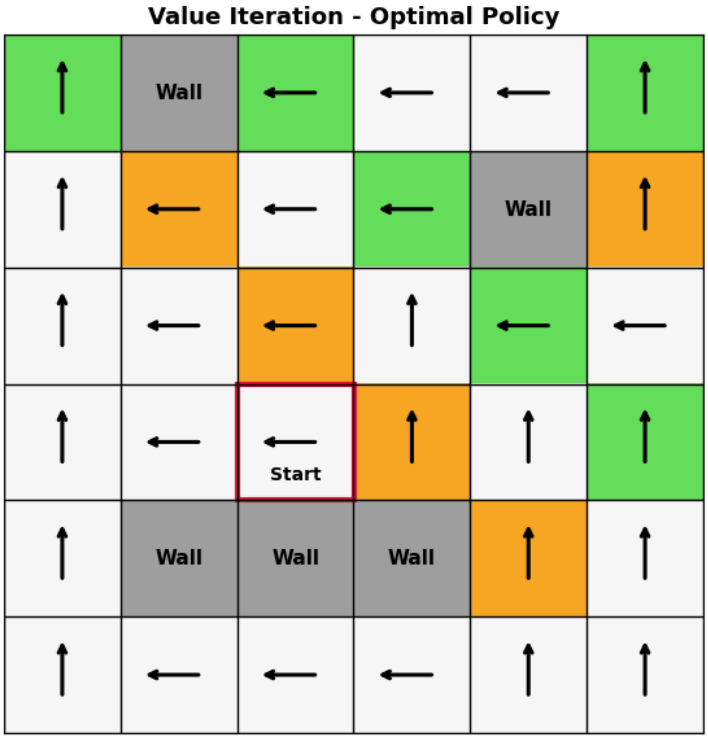


Figure 2: Optimal Policy Obtained Using Value Iteration

1.1.3 Plot of Utilities of All States

Figure 3 shows the final utilities of all non-wall states obtained using Value Iteration. These values represent the maximum expected discounted return from each state using the optimal policy. The states with the highest utilities are the green states or the states with the closest distance to green states, such as the top-left green state achieving the highest utility of 100.00. On the other hand, states that are further from the positive rewards or have higher exposure to brown penalty states have the lowest utility, such as the states in the bottom-right area achieving the lowest utilities. Taking more steps to reach the positive rewards can lower the utility as each white state also has a negative reward of -0.05. Overall, the utilities form a smooth gradient across the maze from lower-valued regions to higher-valued reward-dense regions (Figure 4).

Value Iteration - Optimal Utilities

100.00	**Wall**	**94.97**	94.84	93.59	**93.22**
99.42	**97.85**	95.52	**94.31**	**Wall**	**92.80**
97.95	96.56	**95.21**	94.13	**92.99**	92.72
96.53	95.41	**94.16**	**93.00**	92.74	**91.75**
95.26	**Wall**	**Wall**	**Wall**	**91.41**	91.47
93.86	92.63	91.41	90.21	89.44	90.17

Figure 3: Utilities of All States Obtained Using Value Iteration

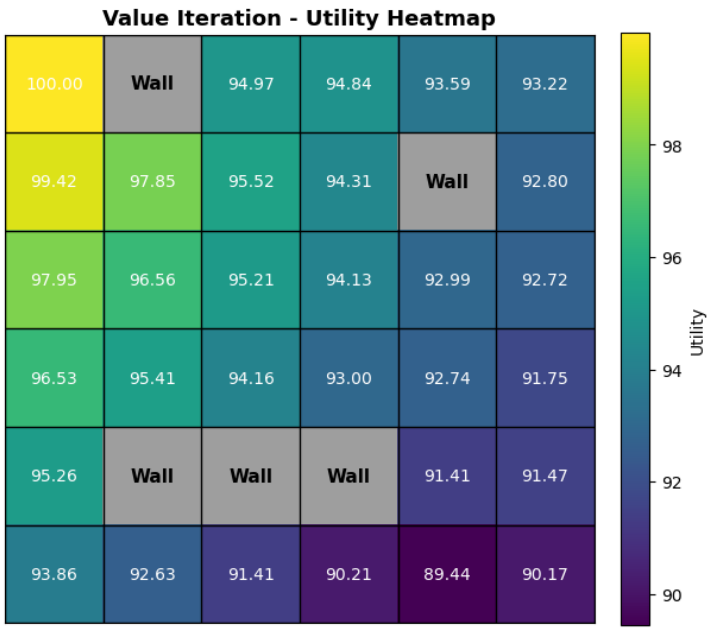


Figure 4: Utility Heatmap Obtained Using Value Iteration

1.1.4 Plot of Utility Estimates Against Number of Iterations

Figure 5 shows the plot of the utility estimates of selected states against the number of iterations. The curves show rapid improvement during the early iterations, followed by gradual improvements from approximately Iteration 200 and plateauing around Iteration 500, indicating convergence towards stable utility values. The algorithm converges at Iteration 1834 to reach a unique optimal solution.

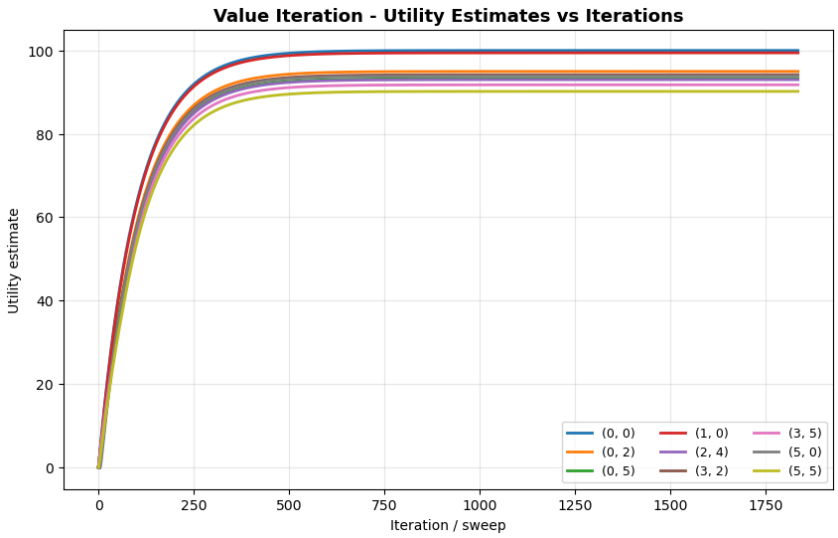


Figure 5: Utility Estimates Against the Number of Iterations for Value Iteration

The plot of Bellman Residual Convergence against the number of iterations on a logarithmic scale is shown in Figure 6. This further shows that as the utilities are updated, the maximum absolute change in utility across all states steadily decreases over time until convergence at Iteration 1834.

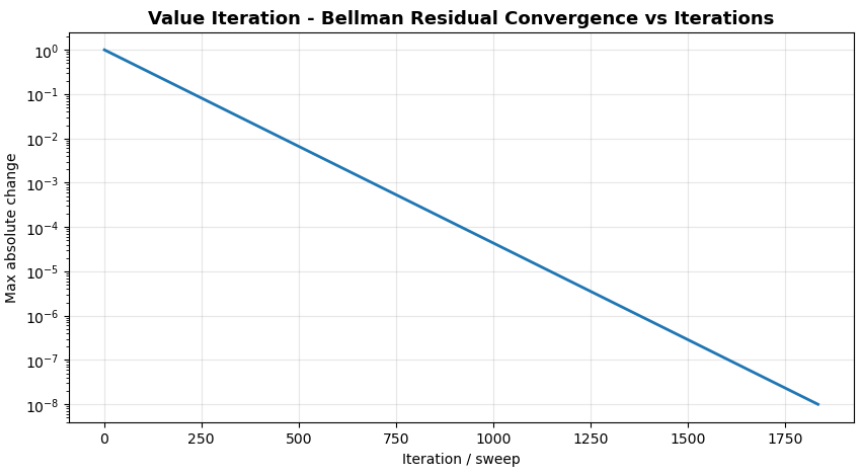


Figure 6: Bellman Residual Convergence Against the Number of Iterations for Value Iteration

1.2 Policy Iteration

1.2.1 Description of Implemented Solution

Policy Iteration is performed by iteratively alternating between two steps: **Policy Evaluation** and **Policy Improvement**. It begins with initialising every non-wall state with the action `Up`. For a fixed policy π, utilities are updated iteratively until the **Policy Evaluation** phase converges. A new greedy policy is then constructed using the evaluated utilities under **Policy Improvement**.

Policy Iteration alternates between:

1. **Policy Evaluation**: Compute utilities for each state under the current policy

   The policy evaluation update is:

$$U_{k+1}^{\pi}(s) = \sum_{s'} P(s' \mid s, \pi(s)) \left[R(s') + \gamma U_k^{\pi}(s')\right] \tag{2}$$

   where:

   - $U_k^{\pi}(s)$ is the utility of state s at evaluation sweep k under policy π
   - $\pi(s)$ is the action chosen by the current policy at state s
   - $P(s' \mid s, \pi(s))$ is the transition probability
   - $R(s')$ is the reward of the next state
   - γ is the discount factor, which is 0.99 in this question

2. **Policy Improvement**: Update the policy greedily based on the expected utilities

   The policy improvement rule is:

$$\pi_{\text{new}}(s) = \arg\max_{a} \sum_{s'} P(s' \mid s, a) \left[R(s') + \gamma U^{\pi}(s')\right] \tag{3}$$

   where:

   - $\pi_{\text{new}}(s)$ is the updated improved action chosen for state s
   - $\arg\max_a$ selects the best action with the highest expected return
   - $P(s' \mid s, a)$ is the transition probability
   - $U^{\pi}(s')$ is the utility of next state s' under the current policy
   - $R(s')$ is the reward of the next state

- γ is the discount factor, which is 0.99 in this question

Similar to Value Iteration, the threshold is set at 1×10^{-8}, and the maximum iterations is set at $10,000$.

1.2.2 Plot of Optimal Policy

Figure 7 shows the optimal policy produced by Policy Iteration.

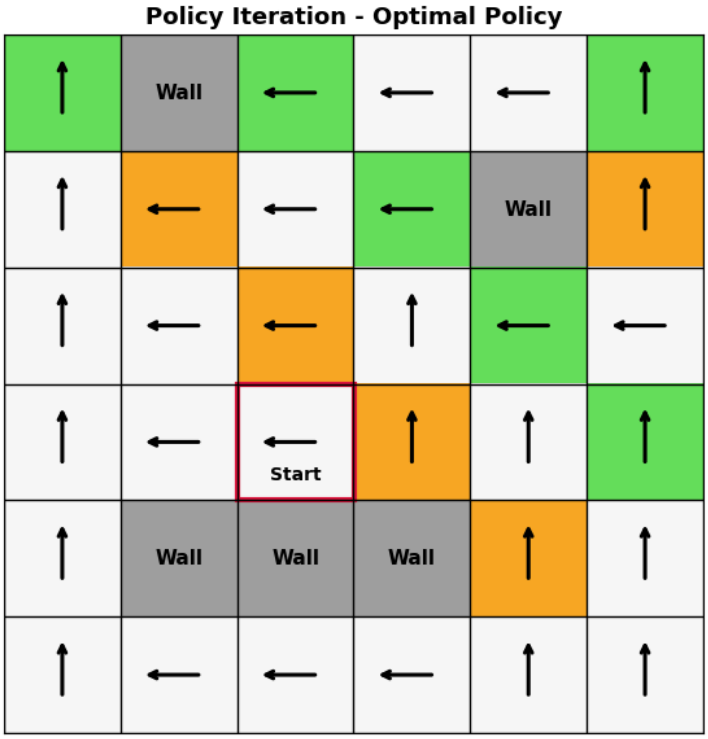


Figure 7: Optimal Policy Obtained Using Policy Iteration

1.2.3 Plot of Utilities of All States

Figure 8 and Figure 9 show the final utilities of all non-wall states obtained using Policy Iteration. The values are approximately identical to those produced by Value Iteration, up to a very small tolerance. This shows that both methods are able to produce the same optimal utility function.

Policy Iteration - Optimal Utilities

100.00	**Wall**	**94.97**	94.84	93.59	**93.22**
99.42	**97.85**	95.52	**94.31**	**Wall**	**92.80**
97.95	96.56	**95.21**	94.13	**92.99**	92.72
96.53	95.41	**94.16**	**93.00**	92.74	**91.75**
95.26	**Wall**	**Wall**	**Wall**	**91.41**	91.47
93.86	92.63	91.41	90.21	89.44	90.17

Figure 8: Utilities of All States Obtained Using Policy Iteration

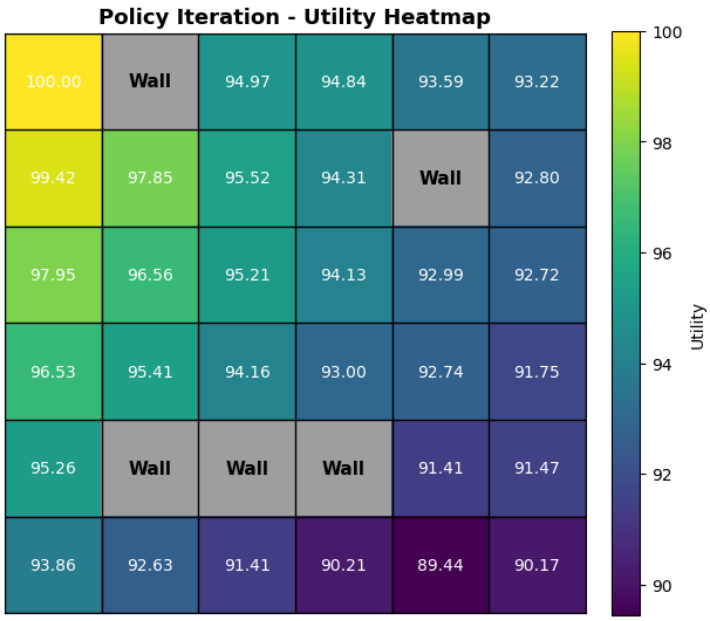


Figure 9: Utility Heatmap Obtained Using Policy Iteration

1.2.4 Plot of Utility Estimates Against Number of Iterations

Figure 10 shows the plot of the utility estimates of selected states against the number of outer iterations. The algorithm converges after only 5 iterations to reach a unique optimal solution, with most of the utility improvements occurring within the first 3 iterations before a plateau is reached.

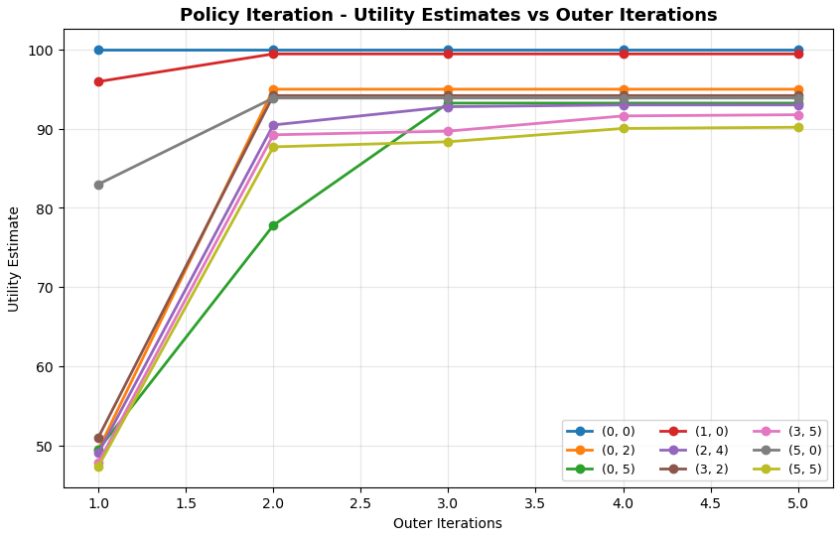


Figure 10: Utility Estimates Against the Number of Outer Iterations for Policy Iteration

Figure 11 plots the number of states whose action was changed after each Policy Improvement step. The number of state changes in each iteration is [19, 8, 3, 3, 0], further showing that the policy adjustments generally occurred in early iterations before convergence at Iteration 5.

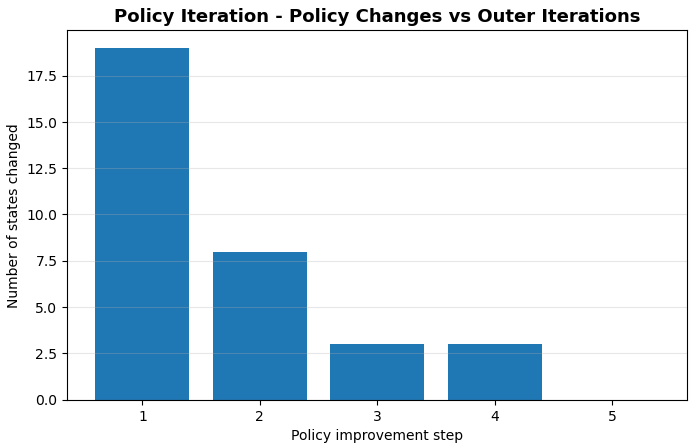


Figure 11: Number of States Whose Action Changed After Each Policy-Improvement Step

However, the plot of the utility estimates of selected states against the number of outer iterations does not fully show the behaviour of the algorithm within each Policy Evaluation step before the next Policy Improvement step occurs. Therefore, we plot the utility

estimates of selected states against the number of evaluation sweeps, displaying how the utilities are updated while the current policy is being evaluated (Figure 12).

Due to the two-step nature of Policy Iteration, unlike Value Iteration, the utility curves are not perfectly smooth. The curves consist of phases with smooth convergence, followed by interruptions highlighted by large jumps in utility. The gradual convergence occurs within each Policy Evaluation step under a fixed current policy. When the policy is improved and updated during Policy Improvement, the next Policy Evaluation phase is evaluated on the target utility function, causing the curves to jump to a different level. The number of sweeps accumulated after each outer iteration was $[1834, 3543, 3710, 3788, 3829]$.

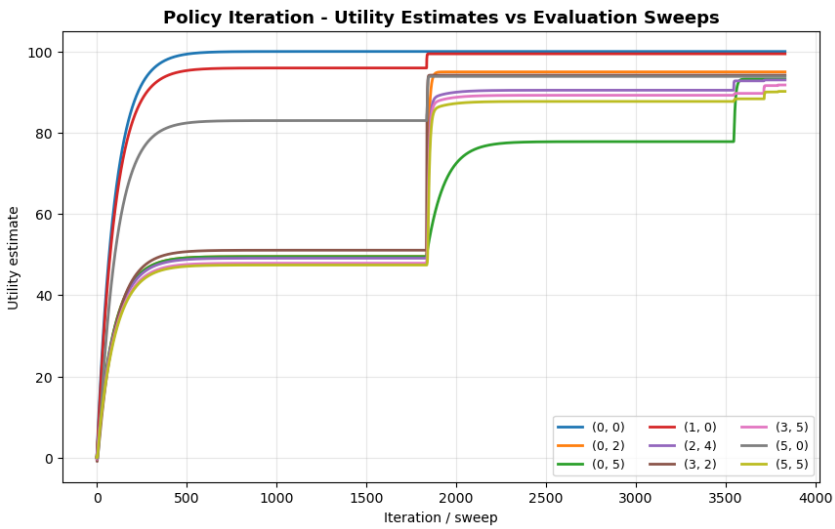


Figure 12: Utility Estimates Against the Number of Evaluation Sweeps for Policy Iteration

This behaviour is also evident in the plot of residual convergence against the number of sweeps (Figure 13), where the residual decreases gradually within each individual Policy Evaluation step, but spikes before decreasing when the utility function is updated using Policy Improvement.

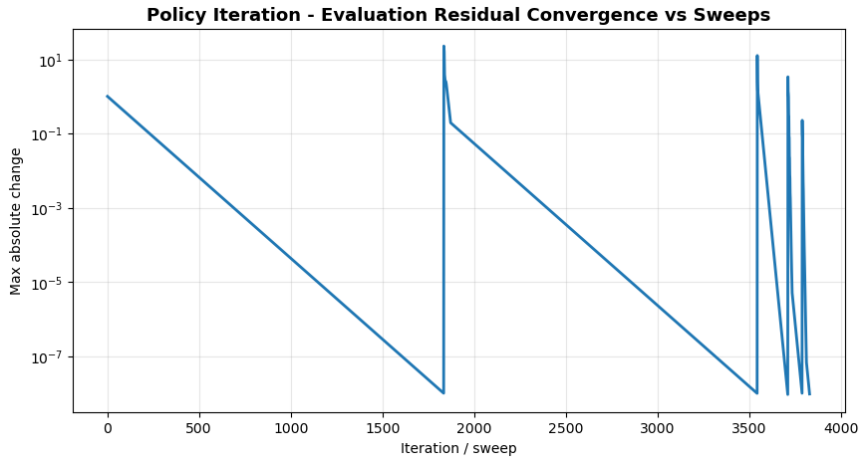


Figure 13: Evaluation Residual Convergence Against the Number of Evaluation Sweeps for Policy Iteration

1.3 Comparison of Results

Overall, both Value Iteration and Policy Iteration generated the same optimal policy and final utilities of non-wall states for the given maze environment. However, their convergence patterns were different. Value Iteration updated the utility of each state through full utility updates of the entire maze, gradually converging at Iteration 1834. On the other hand, Policy Iteration only required 5 outer iterations to converge to the optimal solution. However, at the sweep level, the two-step nature of Policy Iteration caused its utility updates to be less smooth than Value Iteration. The algorithm still required 3829 sweeps to converge.

2 Part 2: Complicated Maze Environments

2.1 Problem Description

This section studies how more complicated maze environments, with different numbers of states and environmental complexities can affect the convergence of both **Value Iteration** and **Policy Iteration**. It also explores the extent of complexity to which the environment can become while still being able to learn the right policy. Section 2 extends Section 1 by applying the same algorithms.

2.2 Methodology

To investigate the effect of maze complexity and number of states on the convergence, the following experimental setups were explored:

1. **Custom Mazes**: Mazes of increasing size and complexity are manually designed, comparing against the given maze in Section 1.

2. **Fixed-Size Wall-Density Experiments**: Mazes are randomly generated at varying wall densities, while keeping the grid sizes fixed. This experiment explores the effect of different wall densities on convergence.

3. **Fixed-Density Size-Scaling Experiments**: Mazes are randomly generated at varying grid sizes, while keeping the wall density fixed. This experiment explores the effect of different grid sizes on convergence.

2.3 Experiments

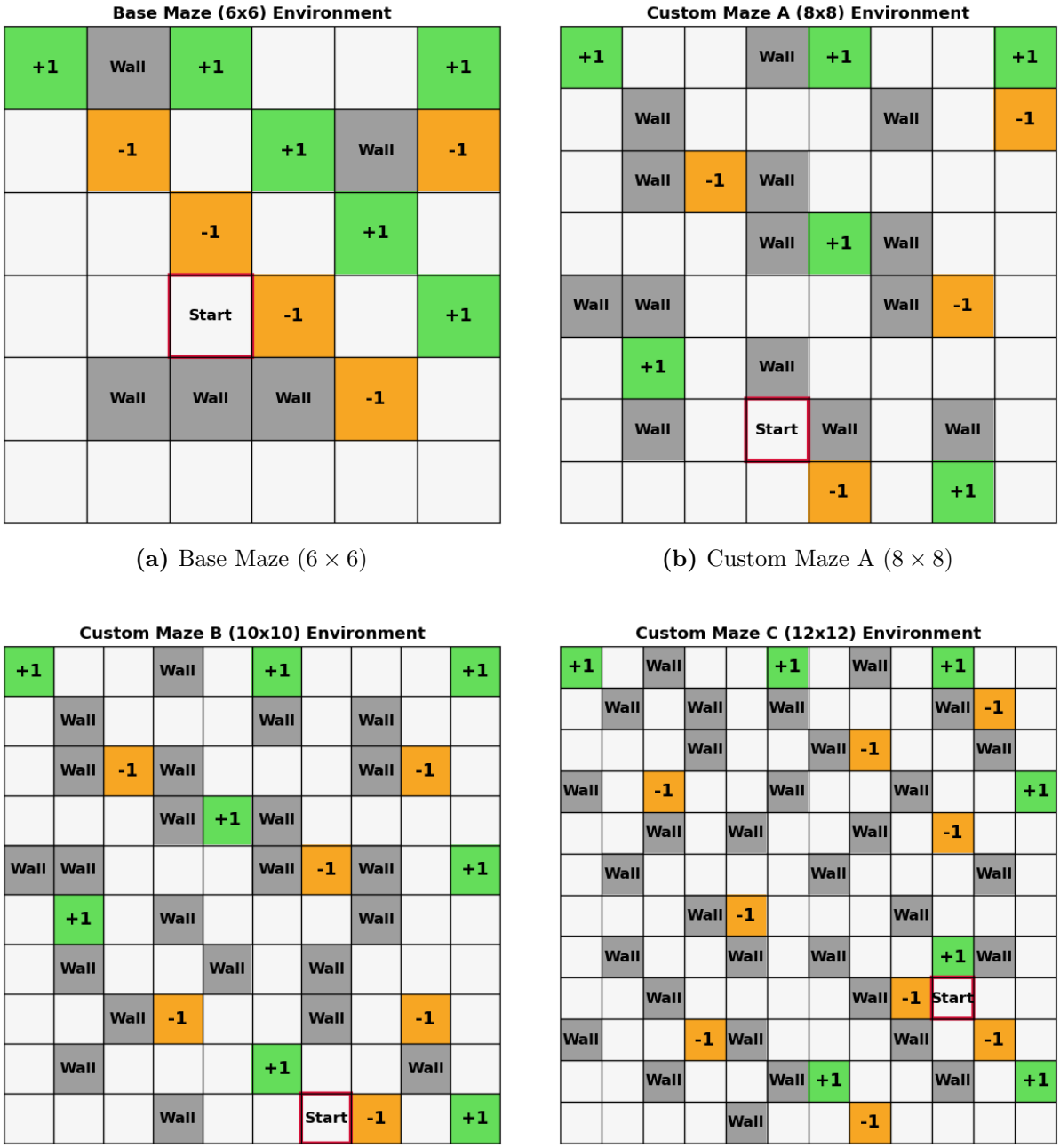
2.3.1 Custom Mazes

Description of the Implemented Solution. The custom mazes used in this experiment are summarised in Table 1, with Base Maze being the given maze used in Section 1.

Table 1: Settings for the manually designed maze experiment.

Maze	Grid Size	Total Cells	Non-Wall States	Start State
Base Maze	6×6	36	31	$(3, 2)$
Custom Maze A	8×8	64	50	$(6, 3)$
Custom Maze B	10×10	100	77	$(9, 6)$
Custom Maze C	12×12	144	111	$(8, 9)$

The figures below show the manually designed environments, visualising the difference in maze size, extent of blockage and route complexity Figure 14.



(a) Base Maze (6×6)

(b) Custom Maze A (8×8)

(c) Custom Maze B (10×10)

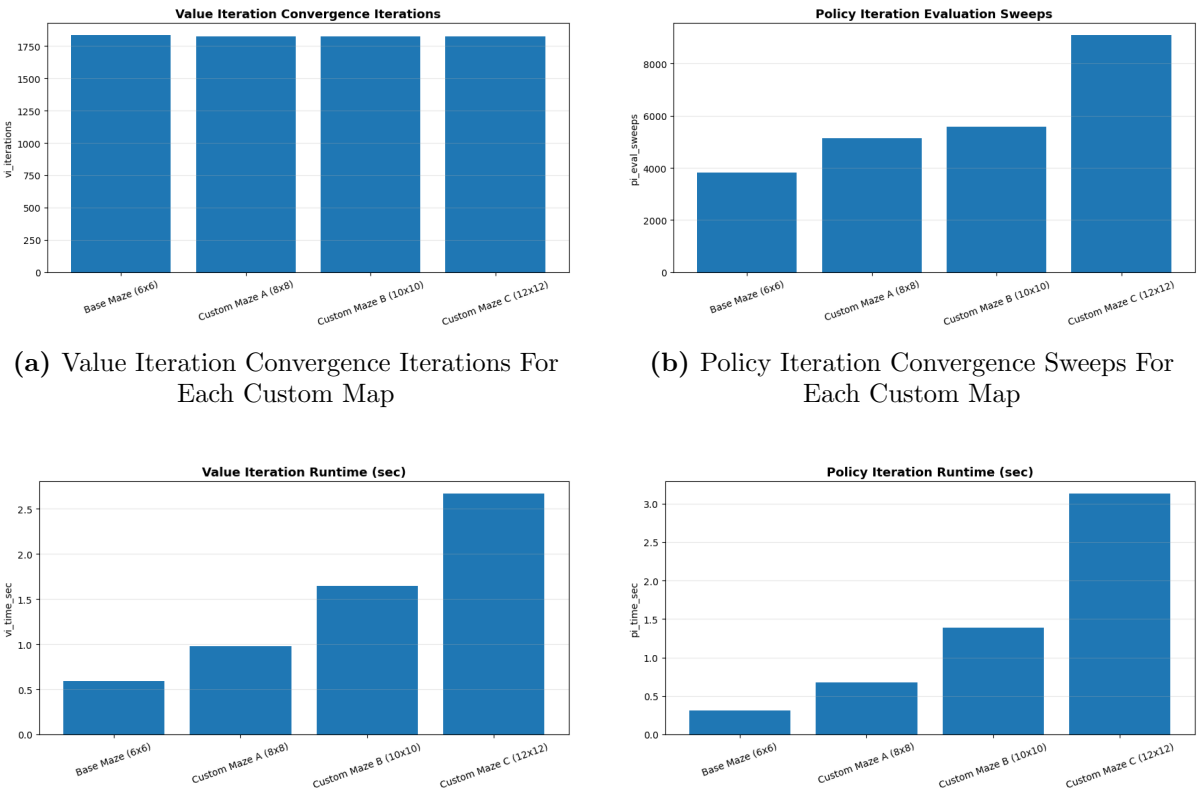
(d) Custom Maze C (12×12)

Figure 14: Overview of the manually designed mazes used in the first Part 2 experiment.

Results and Discussion. The results for the custom mazes are shown in Table 2 and Figure 15. As the number of non-wall states increases from 31 in Base Maze (6×6) to 111 in Custom Maze C (12×12), the runtime per iteration/sweep increases from $0.588s$ to $2.670s$ for Value Iteration and $0.307s$ to $3.131s$ for Policy Iteration. Across the four mazes, the number of iterations till convergence remains unchanged for Value Iteration at approximately 1800 iterations. This highlights that Value Iteration does not necessarily need more updates to converge for larger mazes, but each update requires a longer time. On the other hand, the number of sweeps increased significantly from 3829 to 9088 for Policy Iteration. This suggests that as maze complexity increases, Policy Iteration is more computationally expensive to evaluate and requires more iterations to stabilise than Value Iteration.

Table 2: Summary results for the manually designed maze experiment.

Maze	VI Iter.	PI Eval Sweeps	PI Outer Iter.	VI Runtime (s)	PI Runtime (s)
Base Maze (6×6)	1834	3829	5	0.588	0.307
Custom Maze A (8×8)	1822	5137	5	0.981	0.676
Custom Maze B (10×10)	1822	5587	7	1.649	1.384
Custom Maze C (12×12)	1822	9088	9	2.670	3.131



(a) Value Iteration Convergence Iterations For Each Custom Map

(b) Policy Iteration Convergence Sweeps For Each Custom Map

(c) Value Iteration Runtime For Each Custom Map

(d) Policy Iteration Runtime For Each Custom Map

Figure 15: Overview of Custom Maze Metrics.

Figure 16 and Figure 17 compare the mean utility updates against the number of iterations for the two policies. For both policies, the Base Maze obtained the highest utility, while the most complex and largest Custom Maze C achieved the lowest utility.

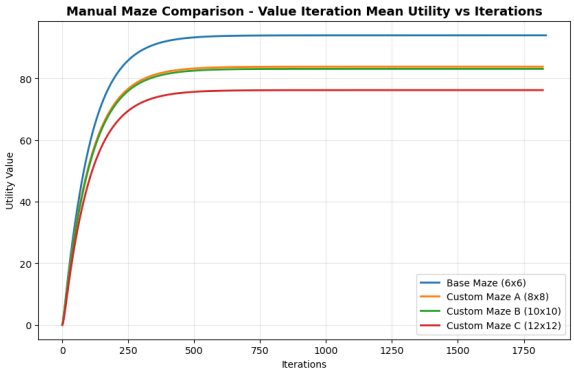


Figure 16: Mean utility against VI iterations for the manually designed mazes.

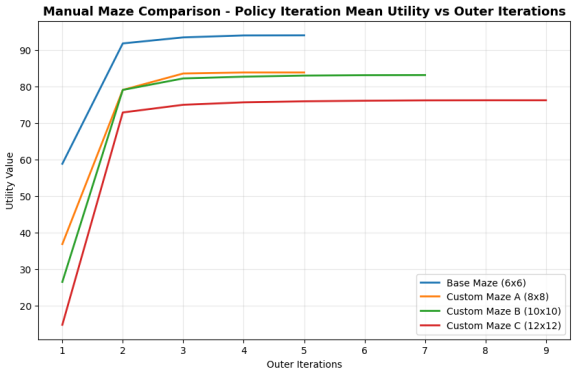


Figure 17: Mean utility against PI outer iterations for the manually designed mazes.

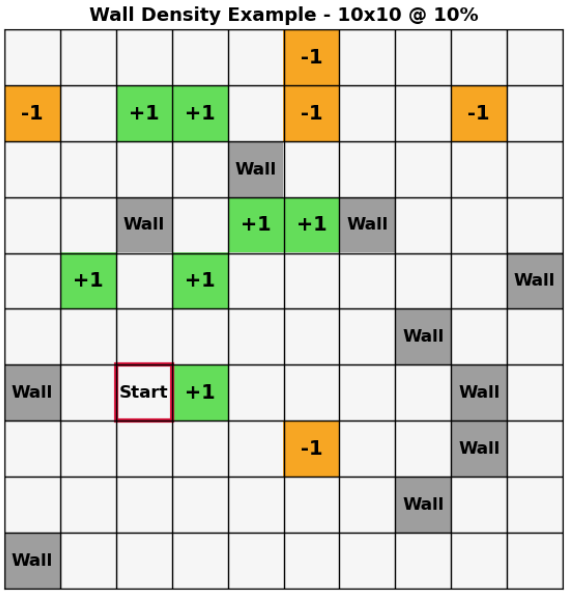
2.3.2 Fixed-Size Wall Density Experiments

Description of the Implemented Solution. This experiment explores the effect of **wall density** on convergence, while keeping the grid size fixed for each setting. This allows isolation of the impact of wall density. For each setting, the grid size remained fixed at 8×8, 10×10, and 12×12, while trials were run for four wall densities 10%, 20%, 30%, and 35%. Each setting was run for 100 trials. Connected mazes were randomly generated for each setting to ensure that all the non-wall states were reachable from the start state. The settings are summarised in Table 3.

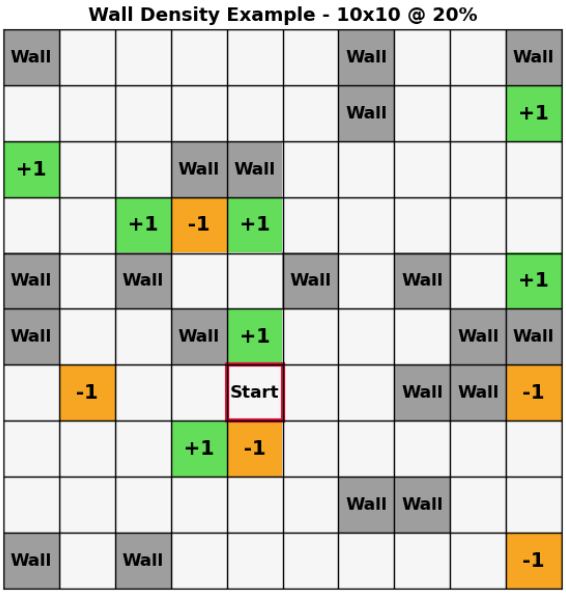
Table 3: Settings for the fixed-size wall-density experiment.

Parameter	Values	Trials per Setting
Grid sizes	$8 \times 8,\ 10 \times 10,\ 12 \times 12$	100
Wall densities	10%, 20%, 30%, 35%	100
Reward placement	Random connected mazes	100

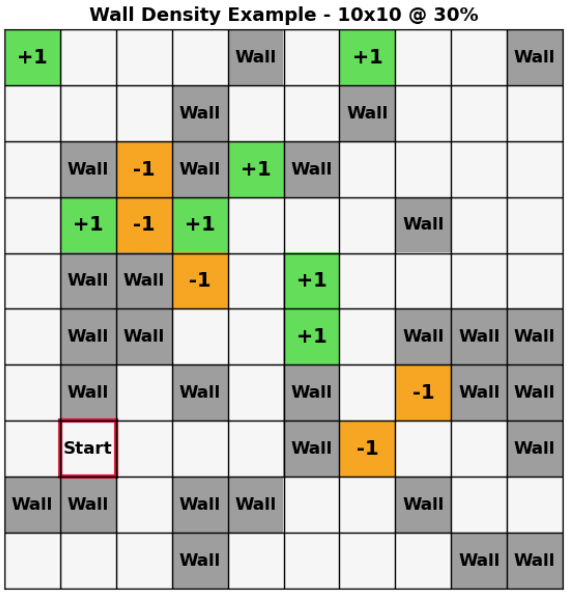
Figure 18 shows examples of randomly generated mazes for each wall density at the 10×10 grid size.



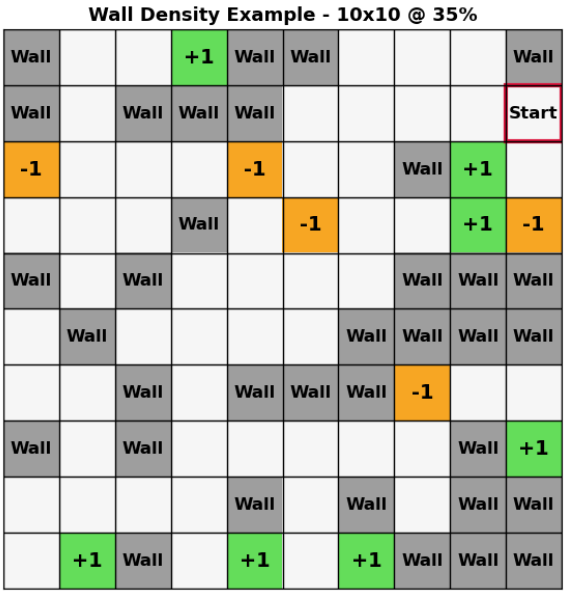
(a) Wall Density Example (10×10)@10%



(b) Wall Density Example (10×10)@20%



(c) Wall Density Example (10×10)@30%



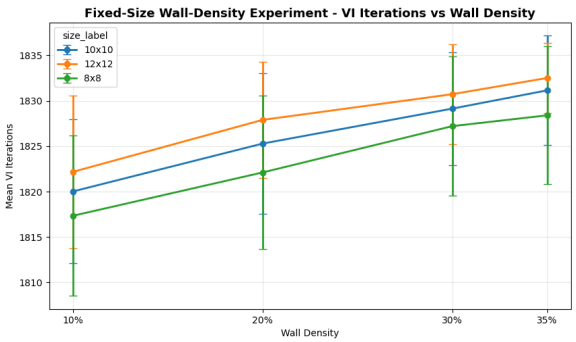
(d) Wall Density Example (10×10)@35%

Figure 18: Representative mazes used in the fixed-size wall-density experiment.

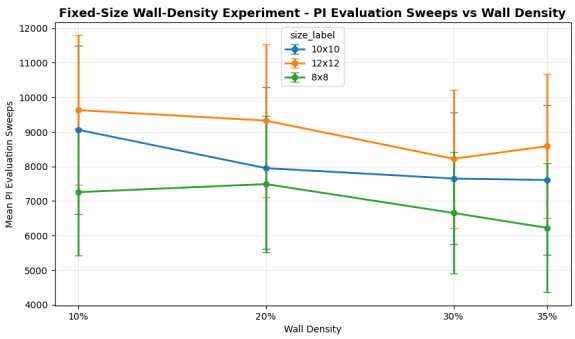
Results and Discussion. Figure 19 shows the plots of iterations against wall density and runtime against wall density for Value Iteration and Policy Iteration. Maze size was the most dominant factor, with the 12×12 mazes consistently achieving the highest runtimes and iterations.

For Value Iteration, the number of iterations till convergence remains approximately the same across all wall densities, remaining within the 1800 to 1840 range. Conversely, Policy Iteration is more sensitive to grid size changes, with the number of sweeps increasing more significantly from the 4500 to 9000 range to the 6200 to 11800 range when grid size increases from 8×8 to 12×12.

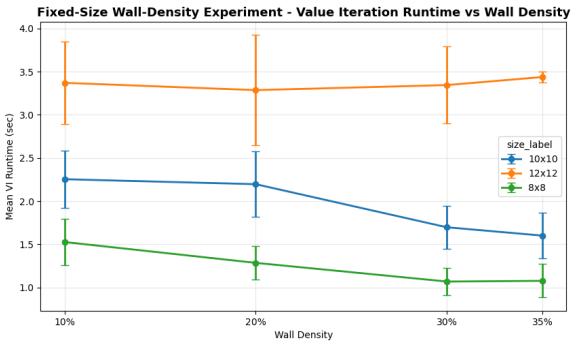
For both algorithms, the runtime is generally stable across all density levels at the same grid size, with only a slight decrease as density increases for the 8×8 and 10×10 mazes as there are fewer non-wall states to be updated. Overall, the grid size affects the iterations/sweeps more than the wall density.



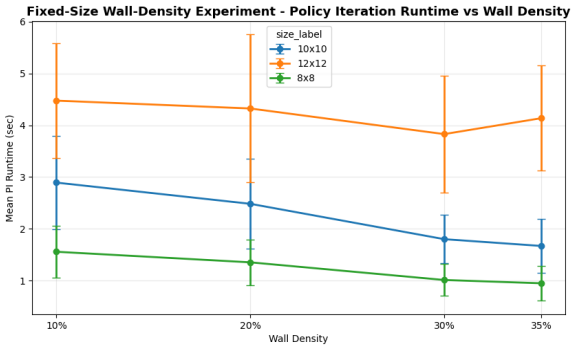
(a) Value Iteration Convergence Iterations For Each Wall Density



(b) Policy Iteration Convergence Sweeps For Each Wall Density



(c) Value Iteration Runtime For Each Wall Density



(d) Policy Iteration Runtime For Each Wall Density

Figure 19: Overview of Fixed-Size Wall-Density Metrics.

Figure 20 and Figure 21 show the plots of utility estimates against iterations for a sample 10×10 maze at each wall density. The wall density does not show a clear correlation with utility value, as the plots show the maze at 35% wall density achieving the highest utility value but the maze at 10% wall density achieving the second highest value. This is

because these are randomly generated mazes, and the utility also depends on other factors like the placement of rewards, walls, and start states. Therefore, it does not imply that higher density improves utility.

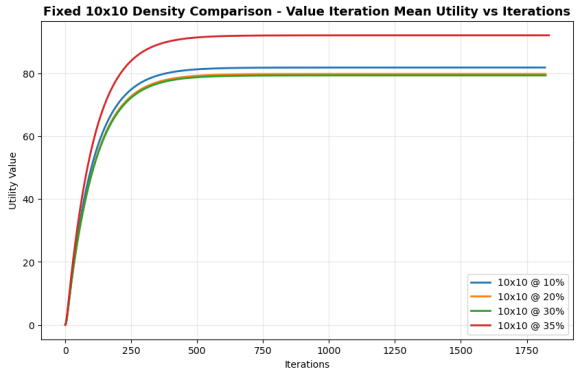


Figure 20: Mean utility against VI iterations for representative 10×10 mazes at different wall densities.

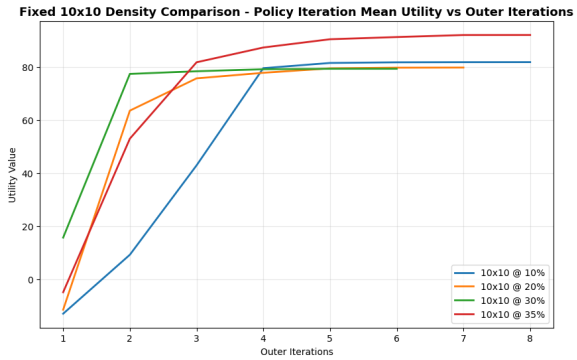


Figure 21: Mean utility against PI outer iterations for representative 10×10 mazes at different wall densities.

Overall, these experiments show that varying wall density does not have a noticeable effect on utility. Section 2.3.3 will further explore the effect of grid size.

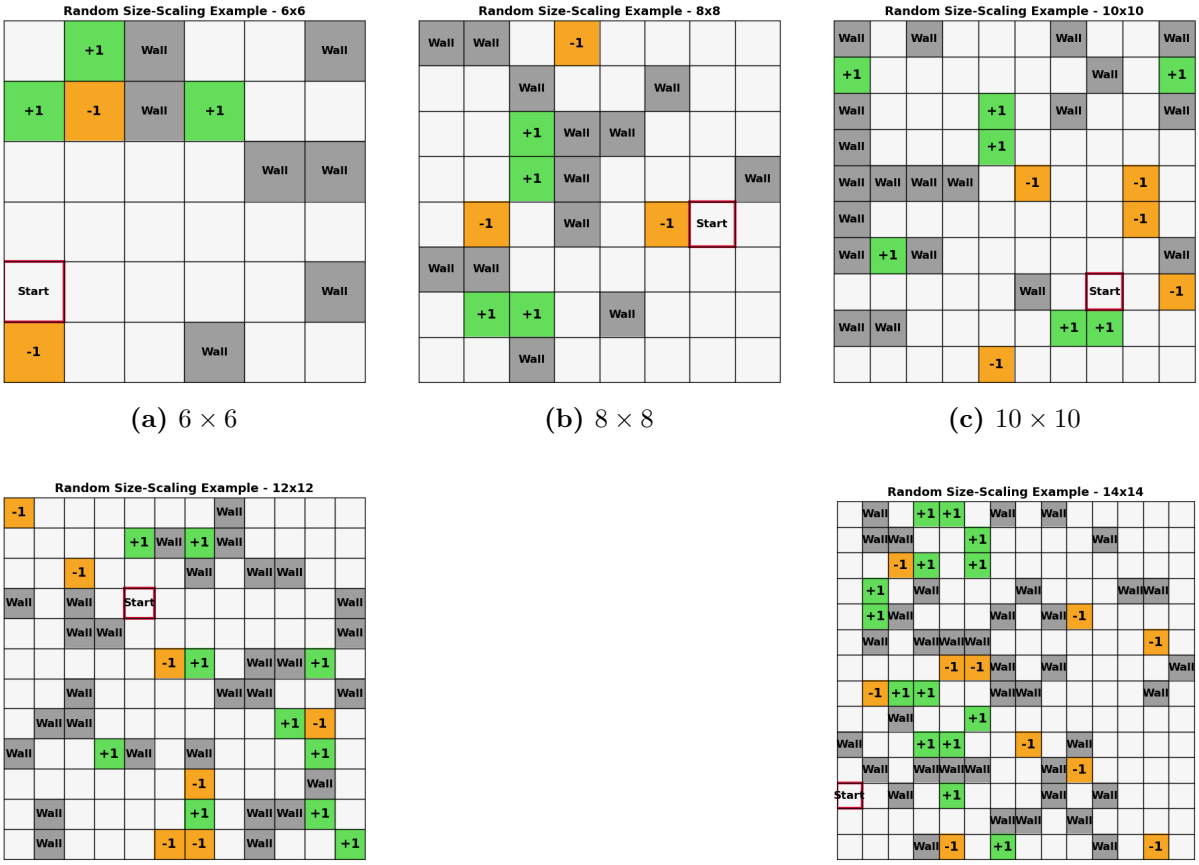
2.3.3 Fixed-Density Size Scaling Experiments

Description of the Implemented Solution. This experiment explores the effect of the **number of states** on convergence by increasing the grid size, while keeping the wall density fixed for each setting. This allows isolation of the impact of grid size. The wall density remained fixed at 20%, while trials were run for five grid sizes 6×6, 8×8, 10×10, 12×12, and 14×14. Each setting was run for 100 trials. Connected mazes were randomly generated for each setting to ensure that all the non-wall states were reachable from the start state. The settings are summarised in Table 4.

Table 4: Settings for the fixed-density size-scaling experiment.

Parameter	Values	Trials per Setting
Grid sizes	$6 \times 6,\ 8 \times 8,\ 10 \times 10,\ 12 \times 12,\ 14 \times 14$	100
Wall density	20%	100
Reward placement	Random connected mazes	100

Figure 22 shows examples of randomly generated mazes for each grid size at 20% wall density.

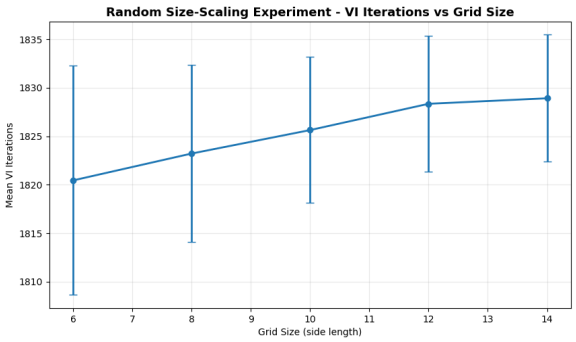


(a) 6×6 **(b)** 8×8 **(c)** 10×10

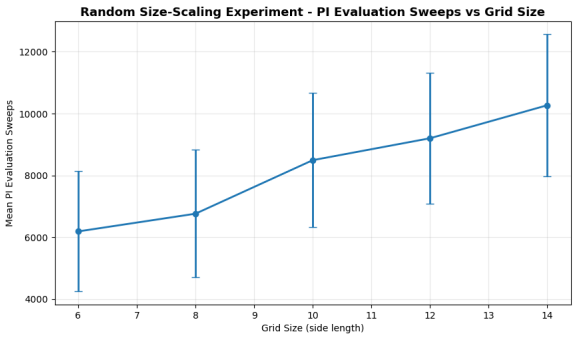
(d) 12×12 **(e)** 14×14

Figure 22: Representative mazes used in the fixed-density size-scaling experiment.

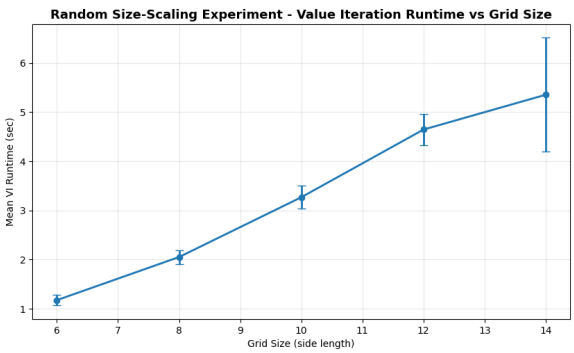
Results and Discussion. Figure 23 shows the plots of iterations against grid size for Value Iteration and Policy Iteration. This further supports the findings in Section 2.3.2. As grid size increases from 6×6 to 14×14, the mean number of non-wall states increases from approximately 30 to 157 states. This increases both the runtime and iterations till convergence for Value Iteration and Policy Iteration, with a more pronounced effect shown in Policy Iteration. For Value Iteration, the number of iterations only increased modestly, staying within the 1800 range. For Policy Iteration, the number of sweeps significantly increases from around 6000 to 10000.



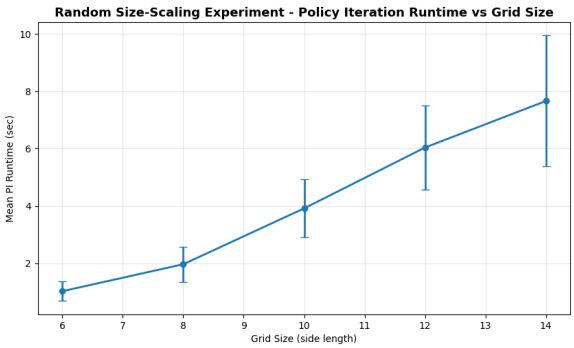
(a) Value Iteration Convergence Iterations For Each Maze Size



(b) Policy Iteration Convergence Sweeps For Each Maze Size



(c) Value Iteration Runtime For Each Maze Size



(d) Policy Iteration Runtime For Each Maze Size

Figure 23: Overview of Fixed-Density Size Scaling Metrics.

Figure 24 and Figure 25 show the plots of utility estimates against iterations for a sample maze at each grid size. Within each algorithm, the curves show a similar trend. However, larger mazes take more outer iterations to converge for Policy Iteration. The sample 8×8 and 10×10 mazes achieving higher utilities might be due to the variability caused by random generation, and not just the sole effect of grid size.

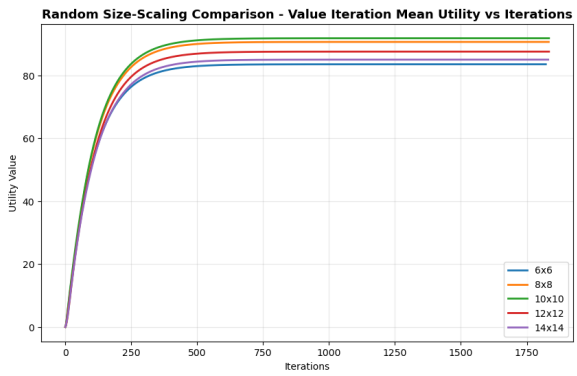


Figure 24: Mean utility against VI iterations for representative mazes of different sizes.

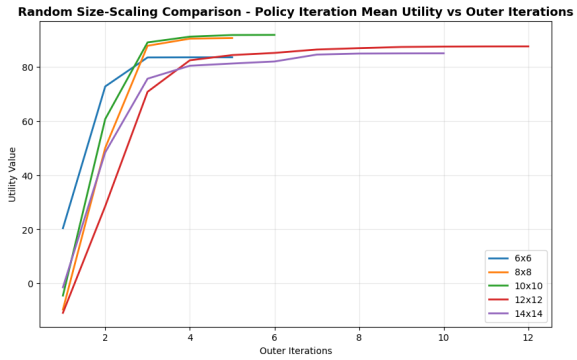


Figure 25: Mean utility against PI outer iterations for representative mazes of different sizes.

Overall, this experiment shows that the number of states is a major determinant of computational cost. Even when wall density is fixed, enlarging the grid substantially increases runtime and Policy Iteration convergence effort, while Value Iteration still maintains a relatively stable iteration count. The stronger conclusion lies in the convergence behaviour, where larger mazes clearly require more runtime and iterations/sweeps, especially for Policy Iteration.

2.4 Conclusion

In conclusion, grid size has a stronger effect on convergence than wall density, with larger mazes attaining a longer runtime and more iterations to reach a stable optimal policy. Grid size also impacts Policy Iteration more than Value Iteration. However, some uncertainty and variability remain as the randomly generated mazes lead to differences in wall and reward placement and route, which also influences the complexity of the maze. Larger mazes were still able to obtain the right policy, with Value Iteration and Policy Iteration matching their utilities up to a small tolerance.